

Trabajo Práctico Integrador:

Carrera	TECNICATURA SUPERIOR en DESARROLLO DE SOFTWARE		
Ciclo Lectivo	2026	Año	3°
Unidad Curricular	Base De Datos II	Docente	Emiliano Gioria
Nombre y apellido	Domingues german, Rios Exequiel	Fecha	24-08-2026
Calificación			

Base De Datos II

Trabajo práctico Integrador parte I

Domingues Germán- Rios Exequiel -

Instituto Tecnológico El Molino (ITEC)

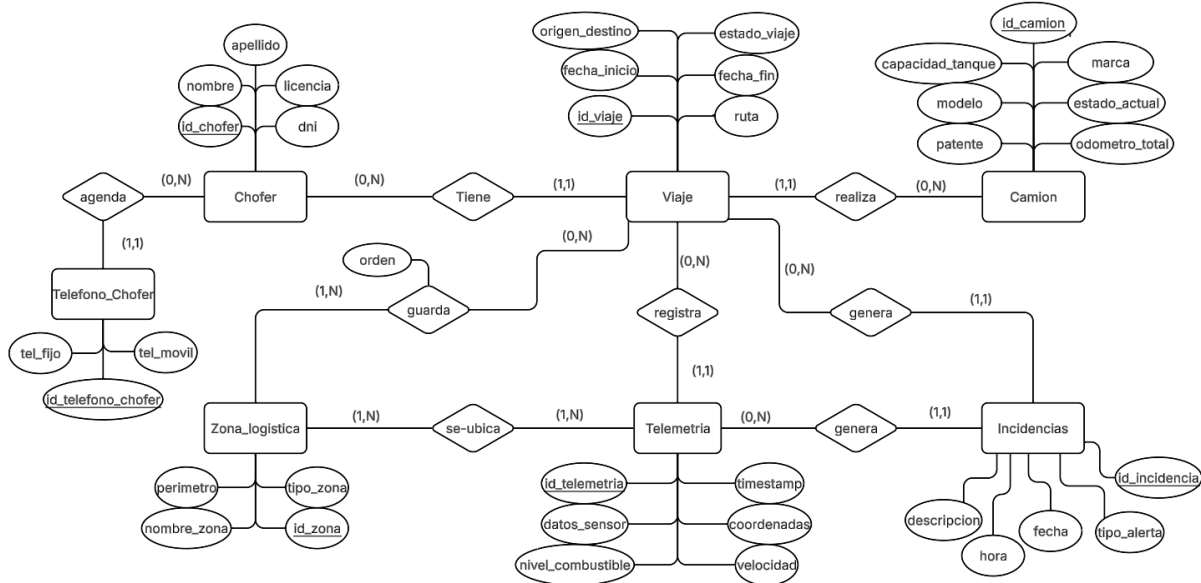
Desarrollo de Software

Emiliano Gioria

Fecha de entrega: 24-08-2026

1. Diagrama entidad relación:	3
2. Reportes de Explain:	3
3. Escenarios criticos	9
4. Escenario donde utilizar Nosql	10
4.1 Análisis de datos para optimización de operaciones	10
4.2 Justificación de jsonb para datos de sensores.	10
5. Aplicación de Sharding(Particionamiento Horizontal).	11
6. Incidencias que pueden ocurrir en una base de datos.	11
7. Rotación de datos.	12
8. Backup.	12

1. Diagrama entidad relación:



2. Reportes de Explain:

- EXPLAIN (ANALYZE, BUFFERS, COSTS) SELECT * FROM vw_ruta_historica;

WindowAgg (cost=2.59..2680.24 rows=101 width=66) (actual time=1.203..2.321 rows=101 loops=1)

Buffers: shared hit=28

-> Incremental Sort (cost=2.59..26.21 rows=101 width=50) (actual time=0.686..0.726 rows=101 loops=1)

Sort Key: telemetria.id_viaje, telemetria."timestamp"

Presorted Key: telemetria.id_viaje

Full-sort Groups: 3 Sort Method: quicksort Average Memory: 28kB Peak Memory: 28kB

Buffers: shared hit=28

-> Index Scan using idx_telemetria_viaje on telemetria (cost=0.14..21.65 rows=101 width=50) (actual time=0.277..0.412 rows=101 loops=1)

Buffers: shared hit=28

Planning Time: 3.184 ms

Execution Time: 2.994 ms

- EXPLAIN (ANALYZE, BUFFERS, COSTS) SELECT * FROM vw_anomalias_combustible;

Subquery Scan on lecturas (cost=29.85..34.93 rows=11 width=135) (actual time=1.578..1.693 rows=1 loops=1)

Filter: ((lecturas.nivel_anterior IS NOT NULL) AND ((lecturas.nivel_anterior - lecturas.nivel_actual) > (lecturas.nivel_anterior * 0.10)) AND ((EXTRACT(epoch FROM (lecturas.fecha_y_hora - lecturas.tiempo_anterior)) / '60'::numeric) <= '5'::numeric))

Rows Removed by Filter: 100
 Buffers: shared hit=24
 -> WindowAgg (cost=29.85..32.12 rows=101 width=115) (actual time=0.977..1.064 rows=101 loops=1)
 Buffers: shared hit=24
 -> Sort (cost=29.85..30.10 rows=101 width=75) (actual time=0.953..0.964 rows=101 loops=1)
 Sort Key: c.id_camion, t."timestamp"
 Sort Method: quicksort Memory: 32kB
 Buffers: shared hit=24
 -> Hash Join (cost=12.86..26.49 rows=101 width=75) (actual time=0.649..0.773 rows=101 loops=1)
 Hash Cond: (v.id_camion = c.id_camion)
 Buffers: shared hit=24
 -> Nested Loop (cost=0.16..13.51 rows=101 width=17) (actual time=0.293..0.392 rows=101 loops=1)
 Buffers: shared hit=23
 -> Seq Scan on telemetria t (cost=0.00..4.01 rows=101 width=17) (actual time=0.030..0.059 rows=101 loops=1)
 Buffers: shared hit=3
 -> Memoize (cost=0.16..0.65 rows=1 width=8) (actual time=0.003..0.003 rows=1 loops=101)
 Cache Key: t.id_viaje
 Cache Mode: logical
 Hits: 91 Misses: 10 Evictions: 0 Overflows: 0 Memory Usage: 2kB
 Buffers: shared hit=20
 -> Index Scan using viaje_pkey on viaje v (cost=0.15..0.64 rows=1 width=8) (actual time=0.025..0.025 rows=1 loops=10)
 Index Cond: (id_viaje = t.id_viaje)
 Buffers: shared hit=20
 -> Hash (cost=11.20..11.20 rows=120 width=62) (actual time=0.113..0.113 rows=11 loops=1)
 Buckets: 1024 Batches: 1 Memory Usage: 9kB
 Buffers: shared hit=1
 -> Seq Scan on camion c (cost=0.00..11.20 rows=120 width=62) (actual time=0.055..0.058 rows=11 loops=1)
 Buffers: shared hit=1
 Planning Time: 1.878 ms
 Execution Time: 2.323 ms

- EXPLAIN (ANALYZE, BUFFERS, COSTS) SELECT * FROM vw_productividad_km;

GroupAggregate (cost=2568.62..2573.92 rows=101 width=132) (actual time=1.565..1.626 rows=9 loops=1)
 Group Key: tramos.id_camion
 Buffers: shared hit=48

```

-> Sort (cost=2568.62..2568.87 rows=101 width=68) (actual time=1.423..1.431 rows=101
loops=1)
  Sort Key: tramos.id_camion
  Sort Method: quicksort Memory: 30kB
  Buffers: shared hit=48
  -> Subquery Scan on tramos (cost=3.68..2565.26 rows=101 width=68) (actual
time=0.716..1.332 rows=101 loops=1)
    Buffers: shared hit=48
    -> WindowAgg (cost=3.68..2564.25 rows=101 width=80) (actual time=0.715..1.316
rows=101 loops=1)
      Buffers: shared hit=48
      -> Incremental Sort (cost=3.68..35.71 rows=101 width=151) (actual
time=0.295..0.408 rows=101 loops=1)
        Sort Key: t.id_viaje, t."timestamp"
        Presorted Key: t.id_viaje
        Full-sort Groups: 3 Sort Method: quicksort Average Memory: 32kB Peak
Memory: 32kB
        Buffers: shared hit=48
        -> Nested Loop (cost=0.30..31.15 rows=101 width=151) (actual
time=0.125..0.252 rows=101 loops=1)
          Buffers: shared hit=48
          -> Index Scan using idx_telemetria_viaje on telemetria t
(cost=0.14..21.65 rows=101 width=147) (actual time=0.057..0.106 rows=101 loops=1)
            Buffers: shared hit=28
            -> Memoize (cost=0.16..0.65 rows=1 width=8) (actual
time=0.001..0.001 rows=1 loops=101)
              Cache Key: t.id_viaje
              Cache Mode: logical
              Hits: 91 Misses: 10 Evictions: 0 Overflows: 0 Memory Usage: 2kB
              Buffers: shared hit=20
              -> Index Scan using viaje_pkey on viaje v (cost=0.15..0.64 rows=1
width=8) (actual time=0.007..0.007 rows=1 loops=10)
                Index Cond: (id_viaje = t.id_viaje)
                Buffers: shared hit=20
Planning:
  Buffers: shared hit=29 read=1
Planning Time: 3.628 ms
Execution Time: 2.024 ms

```

- EXPLAIN (ANALYZE, BUFFERS, COSTS) SELECT * FROM vw_geofencing_alertas;

```

Nested Loop (cost=21.84..60.62 rows=1 width=320) (actual time=0.549..0.761 rows=7
loops=1)
  Join Filter: st_within(t.coordenadas, z.perimetro)
  Rows Removed by Join Filter: 14
  Buffers: shared hit=170 dirtied=1

```

```

-> Unique (cost=21.84..21.85 rows=1 width=102) (actual time=0.351..0.362 rows=7
loops=1)
  Buffers: shared hit=163
-> Sort (cost=21.84..21.84 rows=1 width=102) (actual time=0.351..0.355 rows=70
loops=1)
  Sort Key: v.id_viaje, t."timestamp" DESC
  Sort Method: quicksort Memory: 31kB
  Buffers: shared hit=163
-> Nested Loop (cost=0.30..21.83 rows=1 width=102) (actual time=0.174..0.292
rows=70 loops=1)
  Buffers: shared hit=163
-> Nested Loop (cost=0.16..13.54 rows=1 width=48) (actual time=0.118..0.186
rows=70 loops=1)
  Buffers: shared hit=23
-> Seq Scan on telemetria t (cost=0.00..4.01 rows=101 width=44) (actual
time=0.007..0.034 rows=101 loops=1)
  Buffers: shared hit=3
-> Memoize (cost=0.16..0.65 rows=1 width=8) (actual time=0.001..0.001
rows=1 loops=101)
  Cache Key: t.id_viaje
  Cache Mode: logical
  Hits: 91 Misses: 10 Evictions: 0 Overflows: 0 Memory Usage: 1kB
  Buffers: shared hit=20
-> Index Scan using viaje_pkey on viaje v (cost=0.15..0.64 rows=1
width=8) (actual time=0.008..0.008 rows=1 loops=10)
  Index Cond: (id_viaje = t.id_viaje)
  Filter: ((estado_viaje)::text = 'Activo'::text)
  Rows Removed by Filter: 0
  Buffers: shared hit=20
-> Index Scan using camion_pkey on camion c (cost=0.14..8.16 rows=1
width=62) (actual time=0.001..0.001 rows=1 loops=70)
  Index Cond: (id_camion = v.id_camion)
  Buffers: shared hit=140
-> Seq Scan on zona_logistica z (cost=0.00..12.50 rows=1 width=250) (actual
time=0.007..0.008 rows=3 loops=7)
  Filter: ((tipo_zona)::text = 'Restringida'::text)
  Rows Removed by Filter: 8
  Buffers: shared hit=7 dirtied=1
Planning:
  Buffers: shared hit=39 read=4 dirtied=3
Planning Time: 5.642 ms
Execution Time: 1.136 ms

```

Propuesta de mejora:

1- Índice gist parcial en zona lógicas.

Reporte que solo evalúa zonas con tipo_zona = “restringida”. un índice espacial global sobre perímetro que obligue al motor de búsqueda a revisar las entradas en zonas irrelevantes.

Solución: crear un índice espacial que filtre únicamente los polígonos restringidos.

```
CREATE INDEX idx_zona_restringida_gist  
ON zona_logistica USING GIST (perimetro)  
WHERE tipo_zona = 'Restringida';
```

Impacto: Reduce el tamaño del índice en memoria y permite saltar directamente al algoritmo de intersección espacial indexado, descartando escaneos secuenciales sobre la tabla zona_logistica.

2- Optimizar la última posición mediante un lateral join e índices compuestos

El uso de Distinct on (v.id_viaje) obliga a ordenar miles de registros de telemetría para extraer solo 1 fila por viaje.

Solución: crear un índice compuesto b-tree descendente y reescribir la subconsulta usando cross join lateral

```
CREATE INDEX idx telemetria_viaje_reciente ON telemetria (id_viaje, timestamp DESC);  
-- Consulta optimizada con LATERAL (Index Scan de 1 solo paso por viaje)
```

```
SELECT  
    c.patente,  
    v.id_viaje,  
    z.nombre_zona AS nombre_zona_violada,  
    t_reciente.timestamp AS fecha_hora,  
    ST_AsText(t_reciente.coordenadas) AS ultima_posicion  
FROM viaje v  
JOIN camion c ON v.id_camion = c.id_camion  
CROSS JOIN LATERAL (  
    SELECT t.timestamp, t.coordenadas  
    FROM telemetria t  
    WHERE t.id_viaje = v.id_viaje  
    ORDER BY t.timestamp DESC  
    LIMIT 1  
) t_reciente  
JOIN zona_logistica z ON z.tipo_zona = 'Restringida'  
    AND ST_Within(t_reciente.coordenadas, z.perimetro)  
WHERE v.estado_viaje = 'Activo';
```

Impacto: El plan de ejecución pasa de un sort global costoso a un index scan rápido que lee una única tupla por cada viaje activo.

3- Desnormalización de la última posición en la tabla camión/viaje

En un sistema con ingresos de datos en tiempo real, calcular la posición consultando una tabla histórica con millones de registros de telemetría satura al disco.

Solución: Agregar columnas espaciales ultima_posicion GEOMETRY(Point, 4326) y ultimo_reporte TIMESTAMP directamente en la tabla camion (o viaje), actualizadas automáticamente mediante el trigger de ingesta de telemetría.

Impacto: La vista de Geofencing ya no necesita consultar la tabla telemetría. Realiza una consulta directa a camion WHERE estado_actual = 'En Ruta' contra el índice GiST de zona_logistica, reduciendo el tiempo de respuesta a milisegundos constantes.

- EXPLAIN (ANALYZE, BUFFERS, COSTS) SELECT * FROM vw_estado_sensores;

Append (cost=0.16..31.08 rows=200 width=76) (actual time=0.082..0.251 rows=202 loops=1)

Buffers: shared hit=46

-> Nested Loop (cost=0.16..14.04 rows=100 width=76) (actual time=0.081..0.159 rows=101 loops=1)

Buffers: shared hit=23

-> Seq Scan on telemetria t (cost=0.00..4.26 rows=100 width=115) (actual time=0.026..0.055 rows=101 loops=1)

Filter: (datos_sensor ? 'temp_termografo_c'::text)

Buffers: shared hit=3

-> Memoize (cost=0.16..0.66 rows=1 width=8) (actual time=0.000..0.000 rows=1 loops=101)

Cache Key: t.id_viaje

Cache Mode: logical

Hits: 91 Misses: 10 Evictions: 0 Overflows: 0 Memory Usage: 2kB

Buffers: shared hit=20

-> Index Scan using viaje_pkey on viaje v (cost=0.15..0.65 rows=1 width=8) (actual time=0.002..0.002 rows=1 loops=10)

Index Cond: (id_viaje = t.id_viaje)

Buffers: shared hit=20

-> Nested Loop (cost=0.16..14.04 rows=100 width=76) (actual time=0.015..0.078 rows=101 loops=1)

Buffers: shared hit=23

-> Seq Scan on telemetria t_1 (cost=0.00..4.26 rows=100 width=115) (actual time=0.002..0.019 rows=101 loops=1)

Filter: (datos_sensor ? 'presion_aceite_psi'::text)

Buffers: shared hit=3

-> Memoize (cost=0.16..0.66 rows=1 width=8) (actual time=0.000..0.000 rows=1 loops=101)

Cache Key: t_1.id_viaje

Cache Mode: logical

Hits: 91 Misses: 10 Evictions: 0 Overflows: 0 Memory Usage: 2kB

Buffers: shared hit=20

-> Index Scan using viaje_pkey on viaje v_1 (cost=0.15..0.65 rows=1 width=8) (actual time=0.002..0.002 rows=1 loops=10)

Index Cond: (id_viaje = t_1.id_viaje)

Buffers: shared hit=20

Planning:

Buffers: shared hit=14

Planning Time: 0.832 ms

Execution Time: 0.395 ms

3. Escenarios criticos

1- Asignación concurrente de Camión y Chofer a un Viaje (Aislamiento y Bloqueos)

Descripción: Dos operadores de logística (despachantes) intentan crear un nuevo viaje al mismo tiempo y, por error, seleccionan el mismo camión (ej. patente AF123YZ) o el mismo chofer que figura como "Disponible".

Justificación: Si no existe un mecanismo de aislamiento (Isolation), ambos viajes podrían registrarse en la base de datos asignando recursos que físicamente no pueden estar en dos lugares a la vez. Para evitar esto, es obligatorio iniciar una transacción y utilizar un bloqueo pesimista en la lectura (SELECT ... FOR UPDATE sobre la tabla camión y chofer). Esto bloquea la fila del camión para el primer operador; si el segundo intenta leerla, quedará en espera. Una vez que el primer operador inserta el viaje y actualiza el estado del camión a "En Ruta" (haciendo COMMIT), la transacción del segundo operador se libera, pero ahora verá que el camión ya no está disponible, abortando su asignación (ROLLBACK).

2- Finalización de Viaje y Liberación de Recursos (Atomicidad)

Descripción: Cuando un camión llega a su destino final, el sistema debe realizar tres operaciones simultáneas: 1) Cambiar el estado_viaje a "Finalizado" y registrar la fecha_fin, 2) Actualizar el estado_actual del camión a "Disponible", y 3) Actualizar el odometro_total del camión sumando los kilómetros recorridos.

Justificación: Este escenario depende estrictamente de la Atomicidad ("Todo o nada"). Si el sistema procesa el paso 1 (cierra el viaje) pero ocurre un corte de energía o falla del servidor antes de los pasos 2 y 3, la base de datos quedará en un estado inconsistente (Consistencia vulnerada): el viaje figurará terminado, pero el camión seguirá bloqueado con estado "En Ruta" y su kilometraje desactualizado. Agrupar estos tres UPDATE dentro de una única transacción garantiza que, ante cualquier falla intermedia, se ejecute un ROLLBACK automático, deshaciendo los cambios y manteniendo la integridad de la flota operativa.

3- Reasignación de Unidad por Siniestro o Rotura (Consistencia y Bloqueos)

Descripción: Un camión sufre un desperfecto mecánico en medio de la ruta. El operador debe: registrar una incidencia de tipo "Falla Mecánica", cambiar el estado del camión dañado a "Service", y reasignar el viaje activo a un camión de auxilio que saldrá a buscar la carga.

Justificación: Esta es una operación de alta criticidad que involucra múltiples tablas (incidencias, camión viejo, camión nuevo, y viaje). Se requiere un bloqueo exclusivo sobre la fila del viaje para evitar que el sistema de telemetría, que inyecta datos cada 5 segundos, intente actualizar la posición o los sensores cruzando datos del camión roto mientras se hace el cambio. La transacción asegura la consistencia: la carga no puede quedar "en el aire". El traspaso del viaje del camión A al camión B y el registro del siniestro deben guardarse juntos. Si el camión de reemplazo falla la validación (ej. no tiene capacidad de tanque suficiente), toda la operación de auxilio hace ROLLBACK para que el operador busque otra unidad.

4. Escenario donde utilizar Nosql

Escenario 1: para gestionar documentación del transporte como escaneo o fotos de remitos, permisos de aduana, fotos de siniestros, etc.

Usaría una base de datos nosql para el almacenamiento de estos datos sin forzarlo a ingresarlo en una tabla rígida.

Escenario 2: si la flota de camiones crece emitiendo miles de coordenadas gps, el volumen de escritura podría saturar la base de datos tradicional.

4.1 Análisis de datos para optimización de operaciones

Mantenimiento predictivo: Al analizar el histórico de la telemetría como las caídas de presión de aceite o picos de temperatura, los algoritmos pueden predecir que el camión está a punto de sufrir una falla. Esto permite que el camión pueda ser llevado al taller antes de que se rompa en pleno viaje, ahorrando costos de grúas, cargas perdidas y tiempo

Optimización de rutas y combustibles: Analizando los tiempos de viaje, las horas de tráfico y el consumo de combustible registrados en los reportes, la empresa puede rediseñar las hojas de ruta, por ejemplo un desvío de 20 km consume menos combustible que ir por el camino más corto pero con tráfico abundante o embotellamientos, optimizando así la rentabilidad de cada viaje.

4.2 Justificación de jsonb para datos de sensores.

Utilizar una columna de jsonb es mucho más útil que crear columnas tradicionales, se justifica por la necesidad de su arquitectura para manejar esquemas dinámicos y evitar problemas de diseños. Cada sensor reporta distintas métricas según su marca, modelo o propósito. Si utilizamos un enfoque relacional la tabla requerirá muchas columnas las cuales estarían con muchos datos nulos según el sensor utilizado. lo que degrada el rendimiento de lectura y se desperdicia mucho espacio en el disco.

Al utilizar jsonb resolvemos el problema de raíz, el sistema adquiere esa flexibilidad para ingresar esos datos sin depender de la marca, modelo o propósito del sensor y obteniendo los datos de forma transparente al instante, permitiendo además su rápida consulta mediante la aplicación de índices.

5. Aplicación de Sharding (Particionamiento Horizontal).

Si pasamos de 500 a 50.000 camiones el sistema se va a saturar, el problema no sería por los camiones sino por la gran cantidad de datos que mandan los gps y los sensores cada uno por segundos.

Lo aplicaría en la tabla de telemetría que es donde estaría concentrada la mayor cantidad de datos, también esos generarían muchas incidencias la cual lo podremos aplicar en la tabla incidencias también por si el volumen de datos crece.

Vamos a particionar la tabla telemetría usando el id_camión y en el servidor 1 vamos a guardar los datos de id 1 hasta el 10000, en el servidor 2 de 10001 a 20000 y así sucesivamente.

Esto nos permite distribuir los datos de forma ordenada y no en una sola tabla infinita, quiere decir que si el dueño de la empresa el día de mañana quiere saber por donde anduvo el camión 10500 va a ir directo al servidor 2 haciendo que la búsqueda sea súper rápida.

6. Incidencias que pueden ocurrir en una base de datos.

- Borrado accidental de datos.

Un usuario por accidente, error humano o por un bug en la app ejecuta la orden de borrar sobre viajes históricos o camiones activos, haciendo perder información valiosa.

Esto lo podemos prevenir mediante triggers que bloquean directamente el comando delete en tablas críticas.

- Inserción de datos corruptos.

Un sensor del camión o el gps se rompe o pierde señal, y comienzan a mandar a la base de datos erróneos como que el camión va a 200km/h o que el nivel de combustible es de -70 litros.

Lo podemos prevenir utilizando la cláusula check por ejemplo que la velocidad sea siempre ≥ 0 and ≤ 90 , así la base de datos rechazara la basura antes de que se guarden los datos que realmente nos sirven y si los datos malos ya ingresaron ejecutar un script correctivo como update o delete para limpiar los registros anómalos tal como planteamos en los script de depuración.

- Inconsistencias por caída de servidor a mitad de proceso.

El sistema está comenzando una operación como cerrar un viaje, sumar kilómetros, etc, justo hay un corte de luz y se cuelga el servidor.

La única forma de prevenir que los datos queden es envolver estas operaciones en transacciones acid (begin, commit). Ante la caída la reacción del motor de la base de datos al volver a encender será aplicar automáticamente un rollback.

- Servidor sin espacio.

Después de años acumulando datos de toda la flota, el disco duro de la base de datos llega al 100% de su capacidad, deja de almacenar datos y el sistema deja de funcionar.

Lo prevenimos con una rotación de datos con una técnica de particionamiento. Además de eso podemos eliminar logs antiguos para obtener tiempo para agregar más almacenamiento al servidor (escalabilidad vertical).

- Bloqueos masivos por alta concurrencia.

El gerente está generando reportes pesados que lee toda la información de la tabla histórica, al mismo tiempo muchos camiones están intentando insertar su ubicación actual,

esto genera un embotellamiento donde los procesos se bloquean entre sí y el sistema queda congelado.

La prevención principal es crear índices para que las búsquedas pesadas lean solo lo necesario y terminen rápido para liberar el camino. Si el bloqueo ya ocurrió ingresar como administrador en la base de datos y hacer un kill para matar procesos, para priorizar la inserción de datos.

7. Rotación de datos.

Si mantenemos el histórico de las telemetrías de hace 3 años atrás en la misma tabla donde los camiones están insertando la posición actual segundo a segundo la tabla se vuelve gigantesca. Esto provoca que los índices ocupen demasiada memoria ram. Cuando el motor no puede mantener los índices en memoria tiene que leer el disco físico, lo que vuelve extremadamente lento el monitoreo en vivo de la inserción de datos nuevos.

Por este motivo aplicamos la rotación de datos, movemos la los datos de telemetría viejos a otro almacenamiento trae beneficios directos e indirectos al monitoreo en vivo la tabla principal de postgresQL vuelve a quedar ligera, al tener menos registros las consultas son muchas más rápidas y procesa todo directamente desde la memoria ram.

Por qué elegimos una base de datos de columnas (tipo cassandra) es para mantener la naturaleza de los datos, la telemetría antigua es inmutable los datos antiguos no van a cambiar. Ya no necesitamos las propiedades acid para estos datos.

La base de datos columnares como Cassandra están optimizadas para leer columnas específicamente de forma masiva, si el gerente quiere analizar datos del consumo de combustible del 2024, Cassandra solo lee esta columna rápido mientras que postgres leería la fila entera desperdiciando recursos.

De esta manera estamos aliviando el servidor principal y crítico del negocio, derivando el trabajo pesado de auditoría y estadística histórica a una base de datos secundaria o a una nube mucho más económica.

8. Backup.

Para garantizar la máxima disponibilidad y seguridad de los datos implementamos la regla de 3-2-1 de la siguiente manera.

3. copias de datos.

- Copiar los datos vivos de producción en el servidor principal de postgresQL.
- Hacer un backup de acceso rápido dentro de la empresa.
- un backup de contingencia fuera de la empresa.

2. Medios de almacenamiento distintos.

La copia principal y el primer backup estarán dentro del disco físico local. El backup interno se enviará a un equipo NAS dedicado, separado del servidor de la base de datos, protegiéndonos contra fallas de hardware en el servidor principal.

1. copia en ubicaciones físicas distintas.

Para protegernos ante desastres físicos, el NAS sincronizará una copia cifrada hacia un almacenamiento en la nube ubicado en otra región geográfica.

Dado que la telemetría recibe una flujo constante de datos y no podemos bloquear las tablas por que los camiones siguen reportando en vivo la estrategia de respaldo para los backups sería la siguiente utilizando herramientas de respaldo como pg backrest o barman.

- Backup FULL, se realiza una vez por semana se elige una ventana donde el tráfico de datos sea reducido para minimizar el impacto en el procesador y disco del servidor.
- backup incremental que se realizará de lunes a sábados, como solo copia de los datos nuevos o modificados.
- archivado continuo, adicionalmente a los backups se habilitará el archivado de los archivos wal de postgresql, lo que permitirá que si el servidor explota algún día no perdamos todos los datos, sino que podemos hacer una restauración exacta hasta el momento del desastre.